EXPLICACIÓN DE MI PROGRAMA DE NÓMINA MENSUAL

Programación II - Práctica 11

En este programa registro empleados fijos y por hora para calcular la nómina de un mes. Primero ingreso el número del mes y después utilizo el menú para agregar empleados, consultar la nómina o salir. Los datos quedan en memoria durante la ejecución.

1. Proyecto, paquetes e importaciones

Mi proyecto se llama NominaMensual. Organicé Main.java en el paquete aplicacion y las clases Empleado, EmpleadoFijo y EmpleadoPorHora en modelo. Por eso cada archivo empieza con package aplicacion; o package modelo;. En Main uso import java.util.ArrayList; e import java.util.Scanner; para la lista y la lectura de datos. También importo las clases del paquete modelo.

2. Clases, objetos, instancias e instanciación

Cada clase tiene su propio archivo .java. Empleado contiene los datos comunes, y las clases hijas representan los dos tipos de trabajador. Un objeto es un empleado concreto creado a partir de una clase; ese objeto también es una instancia. En nuevo = new EmpleadoFijo(dpi, nombre, sueldo, bono); hago la instanciación con new. Cada vez que registro a otra persona creo un objeto independiente.

3. Constructor, parámetros, argumentos y this

El constructor EmpleadoFijo(String dpi, String nombre, double sueldoMensual, double bonificacion) recibe los datos iniciales y no tiene tipo de retorno. dpi, nombre, sueldoMensual y bonificacion son sus parámetros. En new EmpleadoFijo(dpi, nombre, sueldo, bono), los argumentos son los valores que envío desde Main. Por ejemplo, para Ana envío su DPI, "Ana Lopez", 4000 y 300. En this.sueldoMensual = sueldoMensual;, this.sueldoMensual es el atributo del objeto y sueldoMensual es el parámetro recibido.

4. Atributos, acceso y encapsulación

Utilizo atributos para guardar los datos de cada empleado. Por ejemplo, private String nombre; y private String dpi; están en Empleado. private limita el acceso directo a la clase que los declara. Consulto sus datos con métodos públicos como obtenerNombre() y obtenerDpi(), que funcionan como getters. No necesito setters porque en este programa no modifico los empleados después de registrarlos. En Main, empleados es una colección privada. En la versión que entrego, el arreglo meses sigue declarado public static; este es un detalle pendiente frente al requisito de tener todos los atributos privados.

5. Abstracción, herencia, super y sobrescritura

Empleado es una clase abstracta y declara public abstract double calcularPago();. No creo objetos directamente de Empleado. EmpleadoFijo y EmpleadoPorHora usan extends Empleado para heredar sus datos comunes. Sus constructores llaman a super(dpi, nombre) para inicializar esos datos, sin duplicarlos. Con @Override implemento calcularPago() en cada hija: el fijo devuelve sueldoMensual + bonificacion y el empleado por hora devuelve horasTrabajadas * pagoPorHora. También implemento obtenerTipo() para mostrar "Fijo" o "Por Hora".

6. String[] meses y acceso por índice

Uso String[] meses para guardar los doce nombres en orden porque la cantidad de meses es fija. numeroMes guarda el número elegido. Consulto meses[numeroMes - 1] porque las posiciones empiezan en cero: con 3 obtengo meses[2], que es Marzo; con 1 obtengo Enero y con 12, Diciembre. Valido el número antes de usarlo como índice. toUpperCase() permite mostrar el nombre en mayúsculas.

7. ArrayList, add(), size(), recorrido y polimorfismo

Uso ArrayList<Empleado> empleados = new ArrayList<>(); porque la cantidad de trabajadores aumenta. empleados.add(nuevo) agrega el objeto después de capturar sus datos, y empleados.size() obtiene la cantidad guardada. En imprimirNomina() recorro la misma lista con for (Empleado e : empleados). Cada referencia e puede representar un empleado fijo o por hora. Al llamar e.calcularPago(), se ejecuta el cálculo de su clase; eso es polimorfismo. Sumo esos resultados con total = total + e.calcularPago();. El reporte no elige la fórmula mediante condiciones por tipo.

8. Métodos, main, resultados y alcance

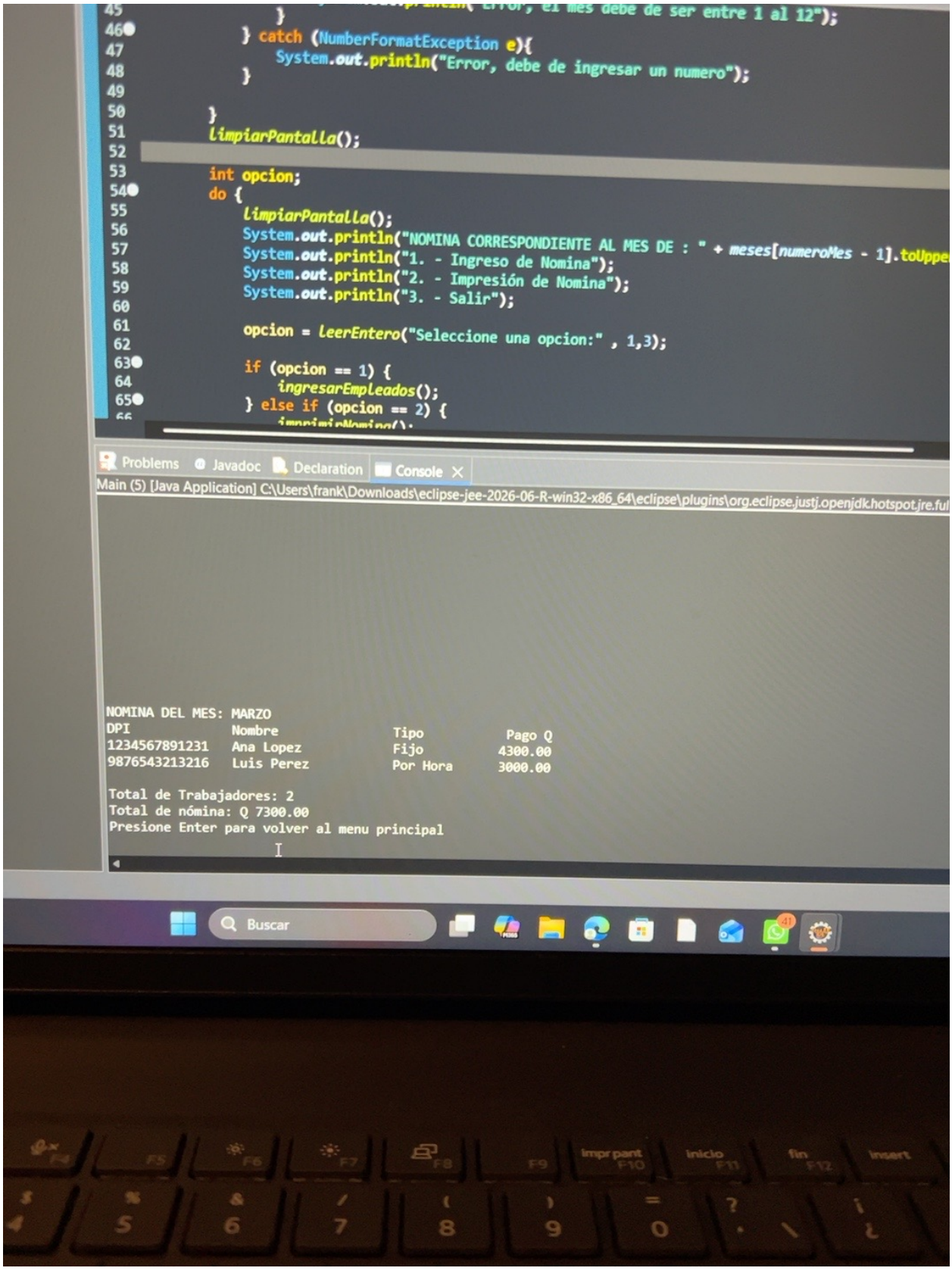
main es el punto de entrada y llama a ingresarEmpleados(), imprimirNomina() y limpiarPantalla(). leerEntero(String mensaje, int min, int max) recibe parámetros y devuelve un int; leerDecimales(String mensaje) devuelve un double. Un atributo como nombre pertenece al objeto y se usa directamente dentro de Empleado. El parámetro mensaje se utiliza dentro del método que lo recibe. La variable local total existe solamente en imprimirNomina(); sueldo existe en el bloque donde capturo al empleado fijo. Las variables locales y los parámetros no llevan private.

9. Pruebas realizadas

Para Marzo registré a Ana Lopez con sueldo de Q4000 y bonificación de Q300: su pago fue Q4300. Registré a Luis Perez con tarifa de Q25 y 120 horas: su pago fue Q3000. El reporte mostró 2 trabajadores y un total de Q7300, como en la sección 11. Los DPI de ejemplo que utilicé fueron 1234567891231 y 9876543213216. También probé Enero sin empleados: mostró 0 trabajadores y Q0.00. Probé meses 13 y 0, texto en el mes, opciones inválidas, DPI incorrectos, nombres vacíos y sueldo negativo o con texto. El programa pidió ingresar otra vez el dato. En otra ejecución elegí Diciembre y registré 120.5 horas para Luis: obtuvo Q3012.50 y el total con Ana fue Q7312.50. Al salir apareció el mensaje de despedida.

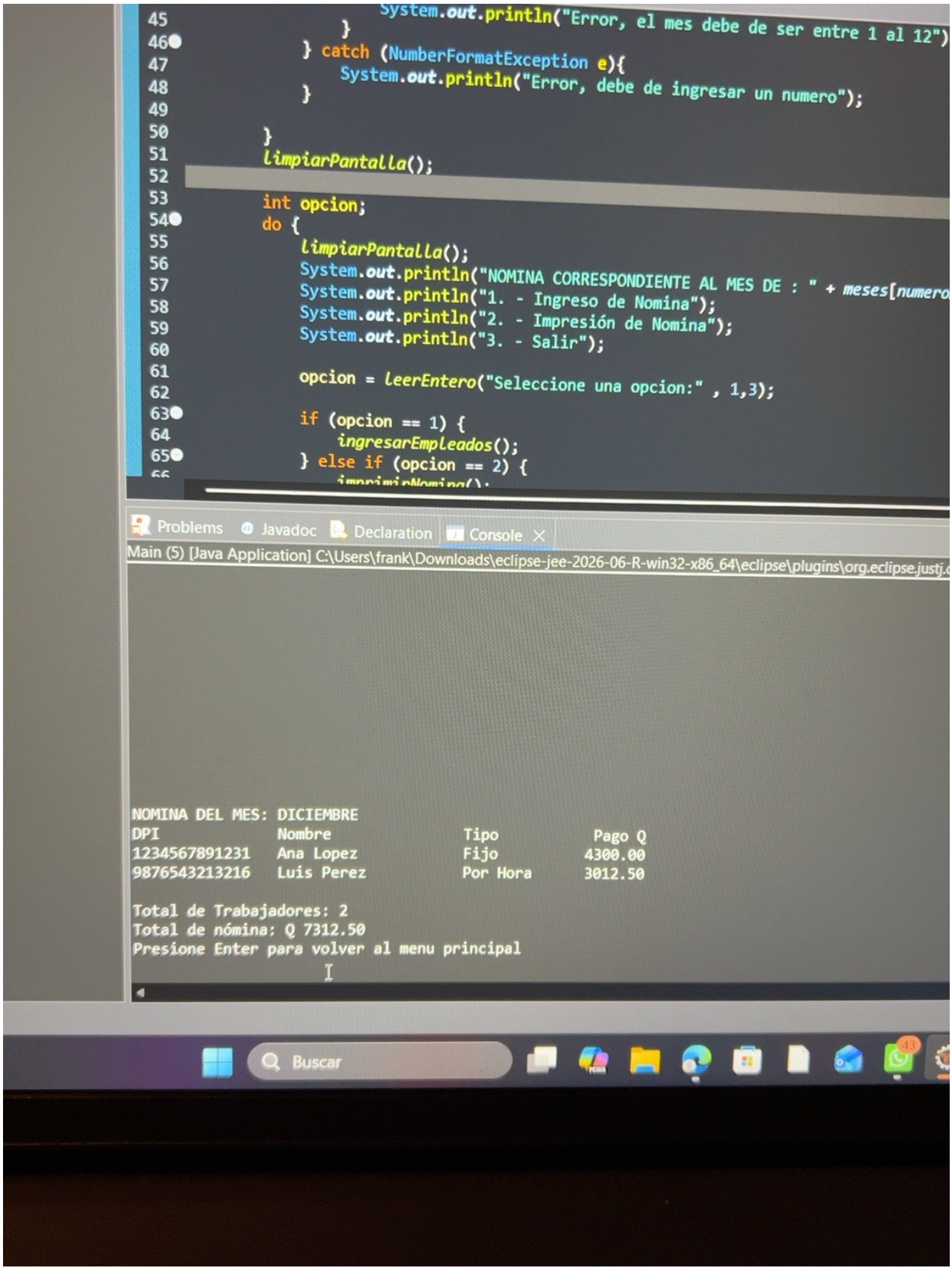
Prueba 1: nómina de Marzo

Ingresé a Ana y a Luis. El reporte mostró Q4300.00 y Q3000.00, con 2 trabajadores y total Q7300.00.



Prueba 2: horas con decimales

En Diciembre probé 120.5 horas a Q25. Luis recibió Q3012.50; con Ana, la nómina sumó Q7312.50.



Prueba 3: entradas inválidas

Probé un DPI corto, otro con letra, nombre vacío y sueldo negativo o con texto. La captura también muestra que esta versión rechaza cero.

